

# Urban Flood Nowcasting System

## System Design Document

---

## 1. Problem Definition

### 1.1 Background

Urban flooding is a major problem in cities with high population density, rapid urbanization, impervious surfaces, limited drainage capacity, and intense rainfall events.

During heavy rainfall, water enters the urban surface as precipitation. A portion of this water infiltrates into the soil, while the remaining water becomes surface runoff. The runoff flows across roads, open areas, and other urban surfaces and eventually reaches stormwater drainage inlets.

When the amount of incoming water exceeds the capacity of the drainage network, water can accumulate on streets and other low-lying areas. Blockages, sedimentation, damaged infrastructure, downstream restrictions, and insufficient drainage capacity can further increase the flooding problem.

Traditional systems may provide rainfall information or historical flood maps, but an effective urban flood management system needs to answer a more useful question:

Where will flooding occur in the next few hours, when will it occur, and how deep will the water become?

The proposed system addresses this problem through short-term urban flood nowcasting.

---

### 1.2 Problem Statement

The system aims to develop a high-resolution, real-time urban flood nowcasting system capable of predicting urban flooding for a lead time of approximately **0–3 hours**.

The system will combine:

- Weather radar rainfall observations
- Rainfall nowcasts
- Digital Elevation Model (DEM)
- Soil characteristics
- Land-use/land-cover information
- Urban imperviousness
- Road and surface information
- Stormwater drainage network
- Hydraulic properties of drainage infrastructure
- Current and historical rainfall conditions
- Current and historical flood observations

These inputs will be processed through rainfall, hydrological, surface-flow, drainage, hydraulic, and flood-simulation components.

The final system will produce spatially and temporally detailed flood predictions that can be displayed through a GIS dashboard and used by a routing system to identify safer routes.

---

## 1.3 Main Objective

The primary objective is:

To predict the location, timing, depth, and severity of urban flooding for the next 0–3 hours by coupling rainfall nowcasting, surface-water flow, and stormwater drainage behaviour.

---

## 1.4 Specific Objectives

The system should:

1. Obtain and process high-resolution rainfall observations.
2. Generate short-term rainfall nowcasts.
3. Convert predicted rainfall into surface runoff.
4. Model surface-water movement using terrain information.
5. Connect surface runoff to stormwater drainage inlets.
6. Represent the drainage system as a graph.
7. Estimate drainage flow and hydraulic capacity.
8. Identify drainage overload and surcharge conditions.
9. Simulate surface-water accumulation.
10. Predict flood depth and timing.

11. Convert predictions into street-level flood information.
  12. Display results through an interactive GIS dashboard.
  13. Provide flood-aware routing through an API.
  14. Use observed flood information for calibration and improvement.
- 

## 1.5 Final Prediction

The final prediction should not simply be:

Flood = YES / NO

Instead, the system should ideally produce:

Location

+

Time

+

Water Depth

+

Flood Severity

+

Duration

For example:

Road: Example Road

Current depth: 0 cm

+30 min: 4 cm

+60 min: 15 cm

+90 min: 31 cm

+120 min: 38 cm

Risk: HIGH

---

## 1.6 Target Users

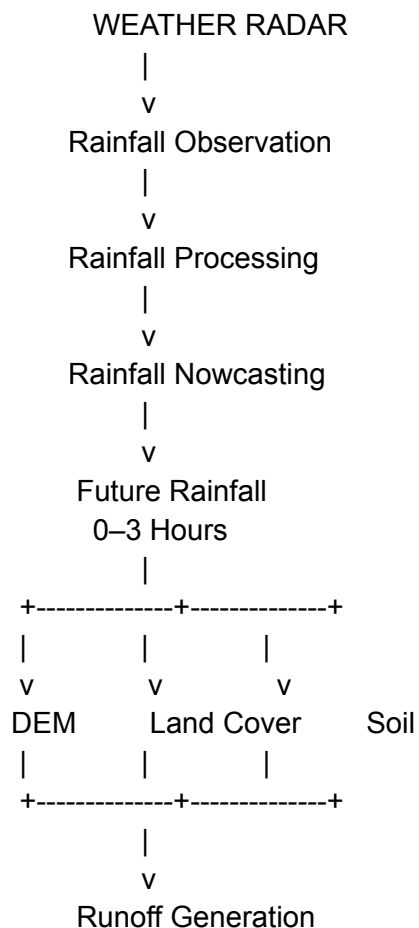
The system is intended to support:

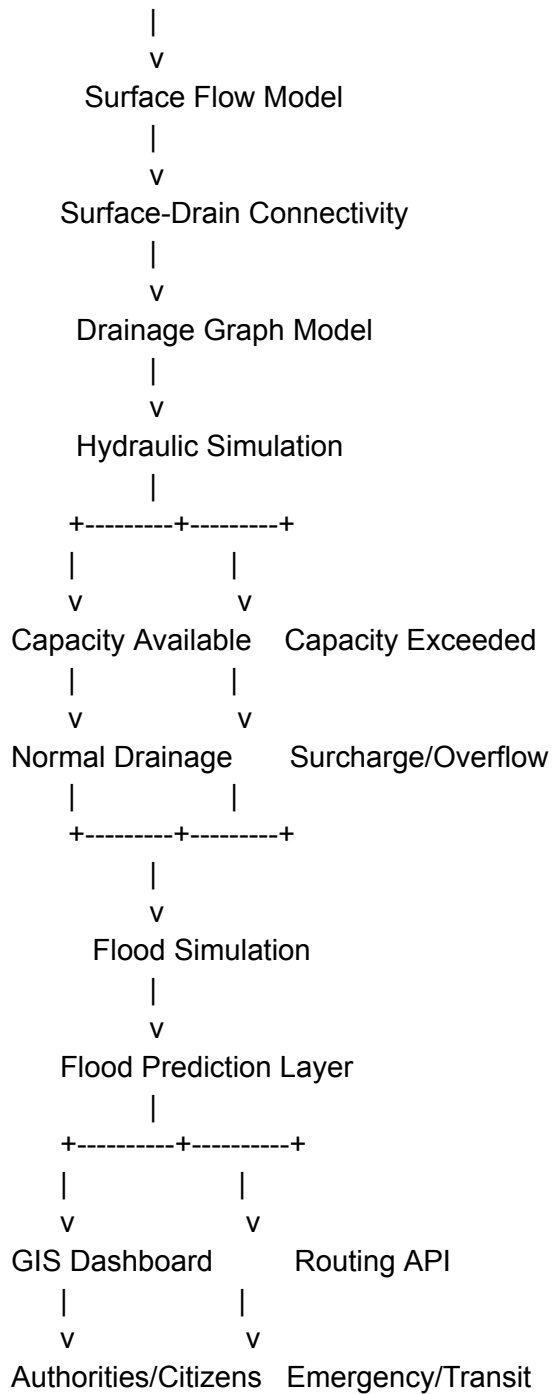
- Disaster management authorities
  - Municipal authorities
  - Emergency response teams
  - Ambulance services
  - Fire and rescue services
  - Police
  - Public transport operators
  - Traffic management authorities
  - Citizens and commuters
- 

## 2. System Architecture

### 2.1 Architecture Overview

The system consists of several interconnected layers.





---

## 2.2 Architecture Layers

### Layer 1 — Data Acquisition

Collects all required input datasets.

Examples:

- Radar rainfall
  - DEM
  - Drainage network
  - Roads
  - Land cover
  - Soil
  - Historical rainfall
  - Historical flood observations
- 

## **Layer 2 — Data Processing**

Converts raw datasets into standardized formats.

Tasks include:

- Cleaning
  - Resampling
  - Coordinate transformation
  - Spatial alignment
  - Temporal alignment
  - Missing-value handling
  - Unit conversion
  - Quality control
- 

## **Layer 3 — Prediction and Simulation**

Contains the main scientific models.

This includes:

- Rainfall nowcasting
  - Runoff generation
  - Surface-flow modelling
  - Drainage-network modelling
  - Hydraulic simulation
  - Flood simulation
-

## Layer 4 — Prediction Processing

Converts model output into useful information.

Examples:

- Flood depth
  - Flood timing
  - Flood severity
  - Affected roads
  - Flooded areas
- 

## Layer 5 — Application Layer

Provides the final user-facing services.

Includes:

- GIS dashboard
  - Flood visualization
  - Alerts
  - Routing
  - APIs
- 

# 3. Data Flow

The complete system data flow is:

Radar Observations

|

v

Rainfall Preprocessing

|

v

Current Rainfall Grid

|

v

Rainfall Nowcasting

|

v

Future Rainfall Grids

|

v

Rainfall + Soil + Land Cover

|

v

Runoff Generation

|

v

Runoff Grid

|

v

DEM + Urban Features

|

v

Surface Flow

|

v

Drainage Inlet Mapping

|

v

Drainage Graph

|

v

Hydraulic Simulation

|

v

Capacity Check

|

+-----+

|

|

Normal

Overloaded

|

|

v

v

Drainage    Surge

|

v

Overflow

+-----+

|

v

Flood Simulation

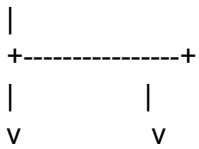
|

v

Water Depth + Time + Location



Street-Level Flood Prediction



GIS Dashboard    Routing API

---

## 3.1 Time Dimension

The system is dynamic.

Instead of predicting only one final state, the model should maintain predictions for multiple future time steps.

Example:

$t$  = current time

$t + 15$  min

$t + 30$  min

$t + 45$  min

$t + 60$  min

...

$t + 180$  min

At every time step, rainfall, runoff, drainage flow, and water accumulation can change.

---

## 3.2 Feedback Data Flow

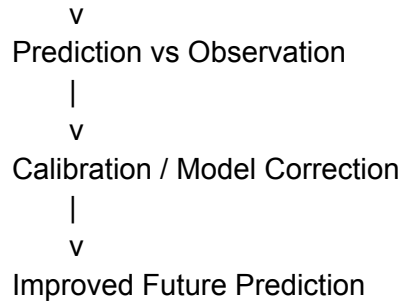
The system can also receive observations after producing predictions.

Flood Prediction



Actual Flood Observation





This feedback mechanism is important for improving prediction accuracy.

---

## 4. Module 1 — Rainfall / Radar

### 4.1 Purpose

The rainfall/radar module provides the system with information about current and recent precipitation.

The radar does not directly provide:

Road A = 25 mm  
Road B = 30 mm

Instead, rainfall radar products generally provide spatial rainfall information represented as a grid or field.

---

### 4.2 Input

Possible inputs include:

- Weather radar observations
  - Radar-derived precipitation products
  - Historical rainfall observations
  - Rain gauge observations where available
-

## 4.3 Processing

The module performs:

1. Data acquisition
  2. Quality checking
  3. Missing-value handling
  4. Coordinate conversion
  5. Spatial alignment
  6. Temporal alignment
  7. Unit conversion
  8. Grid generation
- 

## 4.4 Output

The output is a standardized rainfall field:

Rainfall Grid

Each grid cell can contain rainfall intensity or rainfall accumulation associated with a specific timestamp.

Example:

Grid Cell

-----

Latitude

Longitude

Rainfall

Timestamp

---

## 4.5 Important Consideration

The spatial resolution of the final rainfall grid must be consistent with the actual radar/data resolution.

The system should not claim extremely precise street-level rainfall information if the original rainfall data does not support that resolution.

---

## 5. Module 2 — Rainfall Nowcasting

### 5.1 Purpose

Rainfall nowcasting predicts the future evolution of rainfall over a short time period.

The target is approximately:

0–3 hours

---

### 5.2 Input

The model receives historical and current rainfall fields.

For example:

Rainfall at  $t-60$

Rainfall at  $t-45$

Rainfall at  $t-30$

Rainfall at  $t-15$

Rainfall at  $t$

---

### 5.3 Output

The model predicts:

Rainfall at  $t+15$

Rainfall at  $t+30$

Rainfall at  $t+45$

...

Rainfall at  $t+180$

Each prediction is a spatial rainfall grid.

---

## 5.4 Concept

Rainfall nowcasting needs to capture:

- Movement of precipitation
- Direction
- Speed
- Changes in intensity
- Growth and decay of rainfall systems

Conceptually:

Past Rainfall Fields

|

v

Movement / Evolution Analysis

|

v

Future Rainfall Fields

---

## 5.5 Possible Approaches

The project can evaluate several approaches.

### Baseline

Persistence:

Future Rainfall  $\approx$  Current Rainfall

This is simple and provides a baseline for comparison.

### Optical Flow

Estimate the movement of rainfall patterns.

Rainfall Image

|

v

Motion Estimation

|

v  
Advected Rainfall

## Machine Learning

Possible approaches include:

- CNN-based models
- ConvLSTM
- Temporal CNN
- Transformer-based spatiotemporal models

The final model should be selected based on available data, computational resources, and validation performance.

---

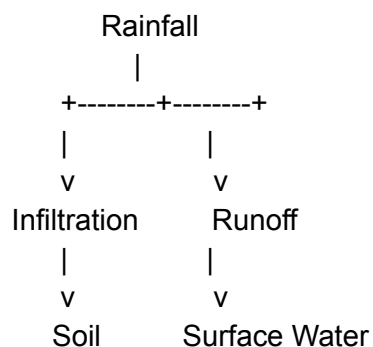
# 6. Module 3 — Runoff Generation

## 6.1 Purpose

Not all rainfall becomes surface water.

The runoff module converts rainfall into the portion of water that contributes to surface flow.

Conceptually:



## 6.2 Factors Affecting Runoff

Runoff depends on:

- Rainfall intensity
  - Rainfall duration
  - Soil type
  - Soil saturation
  - Land cover
  - Imperviousness
  - Slope
  - Antecedent rainfall
- 

## 6.3 Antecedent Conditions

Previous rainfall is important.

For example:

Dry Soil  
+  
10 mm rainfall  
=  
Lower runoff

while:

Already Saturated Soil  
+  
10 mm additional rainfall  
=  
Higher runoff

Therefore, the model should maintain some representation of the current hydrological state.

---

## 6.4 Impervious Surface

Urban areas contain:

- Roads
- Buildings

- Concrete
- Parking areas

These surfaces allow little infiltration.

Therefore:

High Imperviousness



High Runoff

while:

Vegetated / Permeable Area



More Infiltration



Lower Runoff

---

## 6.5 Output

The output is a runoff estimate for each grid cell or catchment.

Runoff(x, y, t)

---

# 7. Module 4 — DEM / Surface Flow

## 7.1 Purpose

This module determines how water moves across the urban surface.

The DEM provides elevation information.

From elevation, the system can derive:

- Slope
- Flow direction
- Flow accumulation

- Depressions
  - Low-lying areas
  - Potential surface drainage paths
- 

## 7.2 Input

- DEM
  - Runoff grid
  - Roads
  - Buildings
  - Urban structures
  - Other relevant surface features
- 

## 7.3 Basic Flow

DEM

|

v

Elevation

|

v

Slope

|

v

Flow Direction

|

v

Flow Accumulation

|

v

Surface Water Path

---

## 7.4 Urban Features

Urban terrain is different from natural terrain.

Important features include:

- Roads
- Buildings
- Walls
- Bridges
- Underpasses
- Low points
- Open drains

These features can change the actual flow path.

---

## 7.5 Output

The module should produce information such as:

- Surface flow direction
  - Surface accumulation
  - Water depth/volume
  - Flow paths
  - Low-lying accumulation areas
- 

# 8. Module 5 — Surface-Drain Connectivity

## 8.1 Purpose

This module connects the 2D surface system to the underground drainage system.

The important question is:

Which surface area sends water to which drainage inlet?

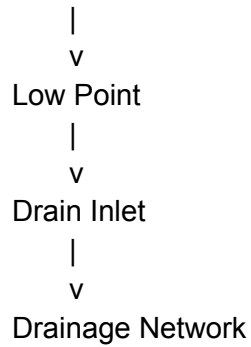
---

## 8.2 Concept

Surface Grid Cell

|  
v

Flow Direction

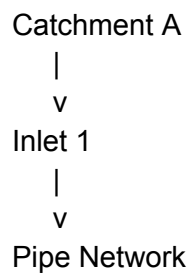


## 8.3 Catchment Concept

A surface area can be divided into drainage catchments.

Each catchment contributes runoff to one or more drainage inlets.

Example:



## 8.4 Output

The output can be represented as:

Surface Cell → Drain Inlet

or:

Catchment → Drainage Node

This relationship is critical for transferring surface runoff into the drainage model.

---

## 9. Module 6 — Drainage Graph

### 9.1 Purpose

The underground drainage network is represented using a graph.

Graph = Nodes + Edges

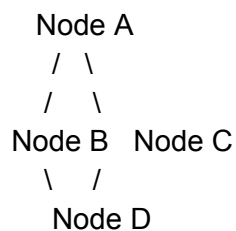
---

### 9.2 Nodes

Nodes may represent:

- Drainage inlets
- Manholes
- Junctions
- Pumping stations
- Outfalls

Example:



---

### 9.3 Edges

Edges represent:

- Pipes
- Channels
- Culverts
- Drainage connections

---

## 9.4 Pipe Attributes

Possible attributes include:

Pipe ID  
Start Node  
End Node  
Length  
Diameter  
Slope  
Material  
Capacity  
Condition

---

## 9.5 Node Attributes

Possible attributes include:

Node ID  
Latitude  
Longitude  
Elevation  
Type  
Incoming Flow  
Water Level

---

## 9.6 Graph Representation

A simplified representation could be:

Node A  
|  
| Pipe 1  
v  
Node B  
|  
| Pipe 2

v  
Node C  
|  
| Pipe 3  
v  
Outfall

The graph allows the system to determine how water propagates through the drainage network.

---

## 10. Module 7 — Hydraulic Simulation

### 10.1 Purpose

The hydraulic module determines how much water can flow through the drainage network and whether the system becomes overloaded.

---

### 10.2 Basic Concept

For every drainage segment:

Incoming Flow  
vs  
Available Capacity

Example:

Incoming Flow = 800 L/s  
Capacity = 1000 L/s

$800 < 1000$

Normal operation

Another case:

Incoming Flow = 1500 L/s

Capacity = 1000 L/s

1500 > 1000

Overloaded

---

## 10.3 Hydraulic Parameters

Important parameters include:

- Pipe diameter
  - Pipe geometry
  - Pipe slope
  - Pipe length
  - Hydraulic radius
  - Roughness
  - Flow velocity
  - Water depth
  - Boundary conditions
- 

## 10.4 Hydraulic Equations

A suitable hydraulic formulation may use equations such as the Manning equation for estimating flow capacity in appropriate conditions.

The exact formulation will depend on the type of drainage infrastructure and the desired level of physical realism.

---

## 10.5 Effective Capacity

The theoretical capacity of a pipe may not equal its real operating capacity.

Factors such as:

- Blockage
- Sedimentation
- Debris

- Structural damage
- Downstream restrictions
- Pump failure

can reduce effective capacity.

Therefore:

Effective Capacity

≤

Theoretical Capacity

---

## 10.6 Output

The hydraulic module can produce:

Pipe Flow  
Water Level  
Pipe Utilization  
Capacity Status  
Surcharge  
Overflow

---

# 11. Module 8 — Flood Simulation

## 11.1 Purpose

This module combines surface water and drainage behaviour to determine where flooding develops.

---

## 11.2 Main Inputs

- Surface runoff
- Surface flow
- Drainage inflow

- Drainage capacity
- Drainage overflow
- DEM
- Current water state

---

## 11.3 Flood Development

Conceptually:

Rainfall

|

v

Runoff

|

v

Surface Flow

|

+-----> Drainage

|

|

|

v

|

Capacity Check

|

|

+---+---+

|

|

|

Normal

|

|

Overflow

|

|

|

|

|

|

|

|

|

|

|

|

|

v

|

Flood State

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

|

---

## 11.4 Dynamic Simulation

Flooding changes over time.

For every timestep:

Rainfall



Runoff



Surface Flow



Drainage



Overflow



Updated Water Depth

Then the updated water depth becomes the starting state for the next timestep.

---

## 11.5 Output

The flood simulation produces:

Water Depth

Water Extent

Flood Start Time

Flood Duration

Flow Direction

Flood Severity

---

# 12. Module 9 — Street-Level Prediction

## 12.1 Purpose

The mathematical simulation may operate on grids, cells, or catchments.

However, users need information about actual roads and streets.

Therefore, model results need to be converted into street-level predictions.

---

## 12.2 Process

Flood Grid

|

v

Road GIS Layer

|

v

Spatial Intersection

|

v

Flood Depth for Road

---

## 12.3 Example

Road A

-----

Predicted depth:

18 cm

Flood starts:

+60 min

Severity:

HIGH

---

## 12.4 Time-Series Output

Each road can contain predictions for multiple time steps.

Road A

Current → 0 cm

+15 min → 1 cm

+30 min → 4 cm

+45 min → 9 cm

+60 min → 18 cm

+90 min → 30 cm

+120 min → 35 cm

---

## 12.5 Risk Classification

The system can classify roads using predefined thresholds.

Example:

LOW  
MODERATE  
HIGH  
SEVERE

The actual thresholds should be defined using domain requirements and validated appropriately.

---

# 13. Module 10 — GIS Dashboard

## 13.1 Purpose

The GIS dashboard provides a visual representation of the model predictions.

The dashboard should allow users to understand:

Where is flooding happening or expected to happen?

When will it happen?

How deep will the water be?

---

## 13.2 Main Features

The dashboard can display:

- City map
- Rainfall radar

- Rainfall forecast
  - Predicted flood areas
  - Water depth
  - Flood severity
  - Drainage network
  - Flooded roads
  - Flood timing
  - Critical locations
  - Safe routes
- 

## 13.3 Time Slider

A time slider can allow users to view:

Now

+15 min

+30 min

+45 min

+60 min

...

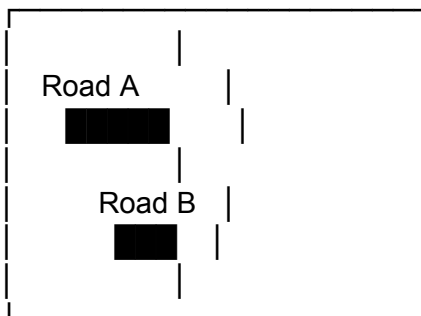
+180 min

---

## 13.4 Visualization

Possible representation:

CITY MAP



Legend:

Low  
Moderate  
High  
Severe

---

## 13.5 Dashboard Users

Different user types can have different views.

### Authorities

Need:

- Flood extent
- Critical locations
- Drainage stress
- Emergency routes

### Citizens

Need:

- Flooded roads
  - Expected depth
  - Safe routes
  - Warnings
- 

# 14. Module 11 — Safe Routing

## 14.1 Purpose

The routing module uses flood predictions to identify safer routes.

A normal navigation system may choose the shortest or fastest route without considering future flooding.

The proposed system adds flood information to the road network.

---

## 14.2 Process

Road Network

+

Predicted Flood Depth

|

v

Flood-Aware Routing

|

v

Safe / Lower-Risk Route

---

## 14.3 Road Risk

Each road segment can be assigned a flood risk or penalty.

Example:

Road A

Flood depth = 0 cm

Risk = Low

Road B

Flood depth = 35 cm

Risk = High

The routing algorithm can avoid or penalize Road B.

---

## 14.4 Emergency Routing

This can be particularly useful for:

- Ambulances
- Fire engines
- Police
- Rescue teams

The system can prioritize:

1. Safety
2. Accessibility
3. Travel time

rather than only shortest distance.

---

## 14.5 Routing API

The system can expose an endpoint conceptually such as:

GET /route

Input:

Start

Destination

Vehicle Type

Departure Time

Output:

Route

Estimated Travel Time

Flood Risk

Affected Roads

Alternative Route

---

# 15. Feedback / Calibration

## 15.1 Purpose

No model will perfectly represent a real city.

Therefore, actual observations should be used to evaluate and improve predictions.

---

## 15.2 Observation Sources

Possible sources include:

- Flood sensors
- Water-level sensors
- Rain gauges
- Municipal reports
- CCTV observations
- Traffic observations
- Crowdsourced reports
- Historical flood maps

---

## 15.3 Prediction vs Observation

The system compares:

Predicted Flood

vs

Actual Flood

Example:

Predicted depth = 20 cm

Observed depth = 25 cm

Error = 5 cm

---

## 15.4 Calibration

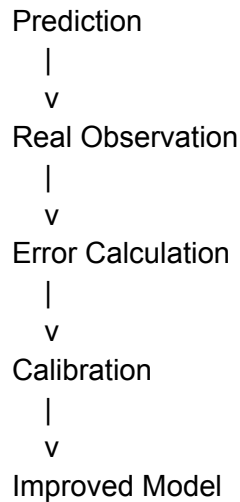
Model parameters can be adjusted based on observed behaviour.

Possible parameters include:

- Runoff coefficients
- Infiltration parameters
- Roughness
- Effective drainage capacity

- Surface-flow parameters
- 

## 15.5 Feedback Loop



This can eventually become a continuous model-improvement system.

---

## 16. Data Requirements

The system requires several categories of data.

### 16.1 Rainfall Data

Required for:

- Current rainfall
- Historical rainfall
- Rainfall nowcasting

Possible data:

Radar rainfall  
Rain gauges  
Historical precipitation

---

## 16.2 DEM

Required for:

- Elevation
- Slope
- Flow direction
- Surface-water routing
- Low-point identification

---

## 16.3 Drainage Network

Required for:

- Drainage graph
- Hydraulic simulation

Potential information:

Manholes  
Inlets  
Pipes  
Channels  
Outfalls  
Diameters  
Lengths  
Elevations

---

## 16.4 Road Network

Required for:

- Street-level flood prediction
- Routing

Potential attributes:

Road ID

Geometry  
Road type  
Length  
Connectivity

---

## 16.5 Land Cover

Required for:

- Runoff estimation
- Imperviousness estimation

Classes may include:

Buildings  
Roads  
Vegetation  
Open land  
Water

---

## 16.6 Soil Data

Required for:

- Infiltration
- Runoff generation

Potential properties:

Soil type  
Infiltration characteristics  
Hydrologic properties

---

## 16.7 Historical Flood Data

Required for:

- Validation

- Calibration
- Model evaluation

Possible information:

Flood location  
Flood extent  
Water depth  
Flood duration  
Time

---

## 16.8 Current Observation Data

Useful for:

- Initial conditions
- Real-time correction
- Calibration

Examples:

Current water level  
Current flood reports  
Sensor observations

---

## 17. Model Inputs

The complete model input can be represented as:

MODEL INPUTS

|                       |  |
|-----------------------|--|
| Rainfall Observations |  |
| Rainfall Nowcast      |  |
| DEM                   |  |
| Land Cover            |  |

|                      |  |
|----------------------|--|
| Soil                 |  |
| Road Network         |  |
| Drainage Network     |  |
| Hydraulic Parameters |  |
| Previous Rainfall    |  |
| Current Water State  |  |

---

## 17.1 Input Categories

### Dynamic Inputs

Change with time:

- Rainfall
- Rainfall forecast
- Water levels
- Flood observations

### Static Inputs

Usually change slowly:

- DEM
- Roads
- Drainage network
- Soil
- Land cover

### Model Parameters

Control model behaviour:

- Runoff coefficients
- Infiltration parameters
- Roughness

- Hydraulic parameters
  - Thresholds
- 

## 18. Model Outputs

The system should produce several levels of output.

### 18.1 Rainfall Output

Future Rainfall Grid

---

### 18.2 Hydrological Output

Runoff per Cell

---

### 18.3 Surface Flow Output

Flow Direction  
Surface Water  
Accumulation

---

### 18.4 Drainage Output

Pipe Flow  
Water Level  
Capacity Utilization  
Surcharge  
Overflow

---

### 18.5 Flood Output

Flood Extent

Water Depth  
Flood Start Time  
Flood Duration  
Flood Severity

---

## 18.6 Street Output

Road ID  
Water Depth  
Flood Risk  
Flood Start Time  
Flood Duration

---

## 18.7 Routing Output

Recommended Route  
Avoided Roads  
Flood Risk  
Estimated Travel Time

---

# 19. APIs

The backend should expose APIs that connect the prediction engine with the dashboard and routing system.

---

## 19.1 Rainfall API

Conceptual endpoint:

GET /api/rainfall/current

Returns:

{

```
"timestamp": "...",  
"rainfall_grid": "..."  
}
```

---

## 19.2 Forecast API

GET /api/rainfall/forecast

Returns future rainfall predictions.

---

## 19.3 Flood Prediction API

GET /api/flood/forecast

Possible parameters:

time  
location  
severity

---

## 19.4 Street Flood API

GET /api/flood/roads

Returns predicted flood conditions for roads.

---

## 19.5 Routing API

GET /api/routing/safe-route

Input:

origin

destination  
departure\_time  
vehicle\_type

Output:

route  
flood\_risk  
affected\_roads  
estimated\_time

---

## 19.6 Observation API

POST /api/observations/flood

Used to submit current flood observations.

---

# 20. Database Design

A spatial database is required to store model inputs, predictions, network information, and observations.

PostgreSQL with a spatial extension such as PostGIS can be considered.

---

## 20.1 Main Entities

Possible tables/entities:

RainfallObservation  
RainfallForecast  
DEM  
LandCover  
Soil  
Road  
DrainageNode

DrainageEdge  
FloodPrediction  
FloodObservation  
ModelRun

---

## 20.2 Rainfall Observation

Conceptually:

RainfallObservation

id  
timestamp  
latitude  
longitude  
rainfall\_value  
source

---

## 20.3 Rainfall Forecast

RainfallForecast

id  
model\_run\_id  
forecast\_time  
grid\_reference  
rainfall\_value

---

## 20.4 Road

Road

id  
geometry  
road\_name  
road\_type  
length

---

## 20.5 Drainage Node

DrainageNode

id  
type  
geometry  
elevation

---

## 20.6 Drainage Edge

DrainageEdge

id  
from\_node  
to\_node  
geometry  
length  
diameter  
slope  
capacity  
roughness

---

## 20.7 Flood Prediction

FloodPrediction

id  
road\_id  
timestamp  
water\_depth  
severity  
confidence  
model\_run\_id

---

## 20.8 Model Run

A model-run entity is useful for tracking individual predictions.

ModelRun

id  
started\_at  
forecast\_start  
forecast\_end  
model\_version  
status

This allows predictions to be associated with a specific model execution.

---

# 21. Technology Stack

The technology stack should be selected based on the separation between the application layer and scientific modelling layer.

---

## 21.1 Backend

Possible:

Node.js  
Express.js

Responsibilities:

- Authentication
  - API management
  - Dashboard APIs
  - Routing API
  - Model orchestration
  - Database access
-

## 21.2 Scientific / ML Layer

Python is suitable for:

Rainfall Nowcasting  
Hydrological Modelling  
GIS Processing  
Hydraulic Simulation  
Flood Simulation  
Machine Learning

Possible framework:

Python  
FastAPI  
NumPy  
Pandas  
GeoPandas  
Rasterio  
PyTorch  
Scikit-learn

The exact libraries should be selected during implementation.

---

## 21.3 Database

Recommended:

PostgreSQL  
+  
PostGIS

PostGIS allows geographic data such as:

- Points
- Lines
- Polygons
- Spatial queries

to be stored and processed.

---

## 21.4 Frontend

Possible:

React

with a GIS mapping library.

Possible mapping technologies include:

MapLibre

Leaflet

OpenLayers

The final choice should depend on project requirements.

---

## 21.5 Deployment

Possible architecture:

Frontend

|

v

Backend API

|

+-----+

|

v

Database    Python Model Service

|

v

ML/Simulation

Containerization can later be handled using Docker.

---

## 22. Evaluation Metrics

Different modules require different evaluation metrics.

---

### 22.1 Rainfall Nowcasting

Possible metrics:

#### **MAE**

Mean Absolute Error.

Measures the average absolute difference between predicted and observed rainfall.

#### **RMSE**

Root Mean Square Error.

Penalizes larger errors more strongly.

#### **Correlation**

Measures similarity between predicted and observed rainfall patterns.

#### **Spatial metrics**

Useful for checking whether the predicted rainfall location and movement match observations.

---

### 22.2 Flood Prediction

Possible metrics include:

#### **Flood Extent Accuracy**

How well predicted flooded areas match actual flooded areas.

#### **IoU**

Intersection over Union:

Predicted Flood Area

$\cap$

Actual Flood Area

-----

Predicted  $\cup$  Actual

## Water Depth Error

Compare:

Predicted depth

vs

Observed depth

using MAE/RMSE.

---

## 22.3 Timing Accuracy

Measure how accurately the system predicts:

When flooding starts

and potentially:

When flooding reaches a threshold

---

## 22.4 Routing Evaluation

Possible metrics:

- Route safety
  - Travel time
  - Distance
  - Number of flooded road segments avoided
  - Emergency accessibility
-

## 23. Limitations

The system will have several limitations.

---

### 23.1 Radar Accuracy

Radar rainfall estimates can contain:

- Measurement errors
- Clutter
- Attenuation
- Missing observations
- Spatial uncertainty

Therefore, rainfall predictions will contain uncertainty.

---

### 23.2 Rainfall Nowcasting Limit

Rainfall becomes increasingly difficult to predict further into the future.

Generally:

Shorter lead time



Higher confidence

Longer lead time



Higher uncertainty

Therefore, the 0–3 hour forecast should be treated as a prediction with uncertainty rather than a guaranteed outcome.

---

### 23.3 DEM Limitations

If the DEM has insufficient:

- Spatial resolution
- Vertical accuracy

then small urban elevation differences may not be represented correctly.

---

## 23.4 Drainage Data Availability

Complete drainage-network information may not be publicly available.

Missing information can include:

- Pipe diameter
- Pipe condition
- Blockage
- Sedimentation
- Underground connectivity
- Current water levels

This can limit hydraulic-model accuracy.

---

## 23.5 Real-Time Blockage Detection

It may not be possible to know the exact blockage level of every drainage segment in real time.

Therefore, blockage may need to be:

- Estimated
  - Inferred
  - Scenario-based
  - Calibrated from historical observations
- 

## 23.6 Current Flood State

If the city already contains standing water before a new forecast begins, assuming:

Initial water depth = 0

may produce incorrect results.

Therefore, current observations or previous model states should be used where available.

---

## 23.7 Computational Cost

High-resolution urban flood simulation can require significant computational resources.

There is a trade-off between:

Resolution

vs

Computation Time

The system must generate predictions quickly enough to be useful for nowcasting.

---

## 23.8 Data Resolution Mismatch

Different datasets may have different:

- Spatial resolutions
- Temporal resolutions
- Coordinate systems
- Accuracy

These datasets must be carefully aligned before being used together.

---

## 23.9 Model Uncertainty

Every stage introduces uncertainty:

Radar

↓

Rainfall Forecast

↓

Runoff

↓

Surface Flow



Drainage



Flood Prediction

Errors can propagate through the entire pipeline.

---

## 24. Possible Improvements

The initial system can be expanded significantly after the core prototype is working.

---

### 24.1 Multi-Source Rainfall Fusion

Combine:

Radar

+

Rain Gauges

+

Satellite

+

Other Observations

to improve rainfall estimation.

---

### 24.2 Advanced Rainfall Nowcasting

After establishing a baseline, advanced models can be introduced:

- ConvLSTM
- CNN-based models
- Transformer models
- Spatiotemporal deep learning

The advanced model should only be adopted if it provides measurable improvement over the baseline.

---

## 24.3 Real-Time Sensors

Deploy sensors at important locations to monitor:

- Drain water level
- Flow
- Road water depth
- Rainfall

These observations can improve real-time state estimation.

---

## 24.4 AI-Based Drainage Condition Estimation

Historical flood observations can potentially be used to infer areas where drainage capacity is lower than expected.

For example:

Repeated Flooding

+

Normal Rainfall

↓

Possible Drainage Problem

This information can be used to estimate effective drainage capacity.

---

## 24.5 Data Assimilation

Current observations can be continuously incorporated into the model.

Model Prediction

+

Current Observation

↓

Updated State



Improved Forecast

This can significantly improve short-term predictions.

---

## 24.6 Uncertainty Prediction

Instead of showing:

Flood depth = 30 cm

the system could eventually show:

Expected depth: 30 cm

Likely range: 24–38 cm

Confidence: 82%

This is more useful for decision-making.

---

## 24.7 Flood-Aware Traffic Management

The system can be connected to traffic-management systems.

Flooded roads could automatically receive:

- Traffic restrictions
  - Detours
  - Warning messages
  - Emergency priority
- 

## 24.8 Emergency Resource Optimization

The system could eventually help authorities decide:

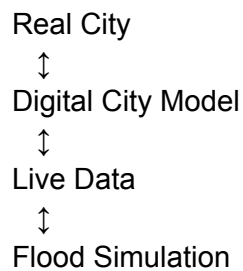
Where should rescue teams be positioned?  
Where should pumps be deployed?  
Which roads should be closed?  
Which areas need priority?

This moves the system from simple prediction toward disaster-response decision support.

---

## 24.9 Digital Twin

A future version could maintain a continuously updated digital representation of the city.

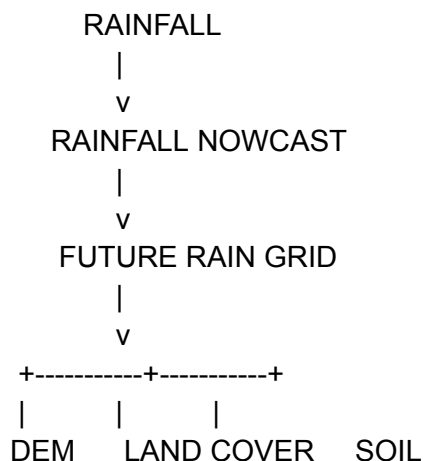


This would allow authorities to simulate different rainfall and drainage scenarios.

---

## 25. Overall System Summary

The complete system can be summarized as:





V                      V  
AUTHORITIES    SAFE ROUTES  
& CITIZENS    & EMERGENCY

The system therefore combines **weather prediction, hydrology, terrain analysis, graph modelling, hydraulic simulation, flood modelling, GIS, and routing** into one integrated urban flood nowcasting platform.

The core objective is not merely to predict rainfall, but to transform predicted rainfall into an actionable answer:

**Where will urban flooding occur, when will it occur, how deep will it become, and which roads should be avoided?**

## **25. Gaps, Challenges & Assumptions**

The following are the major gaps and challenges that must be considered while implementing the Urban Flood Nowcasting System.

### **25.1 Radar Data Gaps**

- Radar observations may contain noise, clutter, attenuation and missing values.
- Radar rainfall estimates may differ from actual ground rainfall.
- Radar coverage and spatial resolution may not be sufficient for every location.
- Radar data must be converted into a consistent spatial and temporal grid.

### **25.2 Rainfall Nowcasting Uncertainty**

- Rainfall nowcasting becomes less reliable as the forecast horizon increases.
- Sudden storm development or dissipation is difficult to predict.
- A single nowcasting model may not work equally well for all rainfall patterns.
- Multiple models or persistence baselines may be required for comparison.

### **25.3 Ground Rainfall Validation**

- Radar rainfall should ideally be validated/corrected using rain-gauge observations.
- Rain gauges provide accurate point measurements but have limited spatial coverage.

- Radar provides spatial coverage but may have estimation errors.

## **25.4 Antecedent Rainfall / Soil Moisture Gap**

- Flood generation depends on previous rainfall, not only current rainfall.
- Soil infiltration capacity changes as the soil becomes saturated.
- Current soil moisture may not be available at sufficiently high spatial resolution.
- Satellite soil-moisture products may be too coarse for street-level modelling.

## **25.5 Land-Cover / Imperviousness Gap**

- Roads, buildings, concrete surfaces and other impervious areas generate more runoff.
- A reliable land-cover or imperviousness layer is required.
- Satellite imagery can help identify impervious and permeable surfaces.
- Cloud cover can limit optical satellite observations.

## **25.6 DEM Limitations**

- DEM resolution and vertical accuracy directly affect surface-flow modelling.
- Small elevation differences may not be represented correctly.
- Buildings, walls, bridges and other urban structures can alter water movement.
- Underpasses and local depressions may require additional modelling.

## **25.7 Surface-to-Drain Connectivity Gap**

- Knowing the drainage network alone is not enough.
- The model must determine which surface areas drain into which inlets/manholes.
- Simply assigning runoff to the nearest drain may produce incorrect results.
- Catchments and surface flow paths need to be connected to drainage nodes.

## **25.8 Drainage Network Data Gap**

- Complete underground drainage GIS data may not be publicly available.
- Pipe diameter, length, slope, material and condition may be missing.
- Manhole/inlet locations may be incomplete or inaccurate.
- Different authorities may maintain data in different formats.

## **25.9 Drain Blockage Data Gap**

- Real-time blockage percentage is usually difficult to obtain.
- Blockages can change during rainfall events because of debris, garbage and sediment.
- Historical maintenance records may be incomplete.
- Blockage may therefore need to be represented using estimated/scenario-based effective capacity.

## **25.10 Hydraulic Parameter Gap**

- Theoretical pipe capacity may differ from actual effective capacity.
- Sedimentation, damage, blockage and downstream restrictions can reduce capacity.
- Hydraulic parameters may need calibration using historical observations.

## **25.11 Current Flood-State Gap**

- The model cannot always assume that the city starts with zero water.
- Existing water on roads/drains affects the next prediction.
- Current water levels may need to come from sensors, cameras, reports or other observations.
- The model should ideally support data assimilation/correction using current observations.

## **25.12 Historical Flood Data Gap**

- Accurate historical flood-depth observations may be limited.
- Historical datasets may contain only flood/no-flood information.
- Flood observations may have inconsistent timestamps and locations.
- Historical events are required for calibration and validation.

## **25.13 Street-Level Prediction Gap**

- Input datasets may have coarser resolution than individual streets.
- The system should not claim precision greater than the underlying data supports.
- Road-level depth should be derived carefully from the flood grid and road geometry.
- Local drainage structures may cause conditions to vary significantly within a single road.

## **25.14 Temporal Resolution Gap**

- Different datasets may have different update frequencies.
- Radar, satellite, sensors, drainage observations and traffic data may not be synchronized.
- A common simulation timestep is required.
- The system must maintain correct timestamps and handle missing observations.

## **25.15 Data Integration Gap**

- Datasets may use different coordinate reference systems.
- Spatial resolutions and geographic boundaries may differ.
- Units and formats may be inconsistent.
- A preprocessing and standardization layer is required before modelling.

### **25.16 Real-Time Processing Gap**

- The complete pipeline must process new rainfall observations continuously.
- Running rainfall nowcasting, runoff, surface flow, drainage hydraulics and flood simulation together may be computationally expensive.
- Model complexity must be balanced with the required real-time response.

### **25.17 Uncertainty Propagation**

- Errors in rainfall nowcasting propagate into runoff predictions.
- Runoff errors propagate into drainage simulation.
- Drainage uncertainty propagates into flood-depth predictions.
- The final system should ideally communicate prediction confidence or uncertainty.

### **25.18 Model Calibration Gap**

- Model parameters cannot simply be assumed to be correct.
- Historical rainfall and flood events should be used for calibration.
- Parameters such as infiltration, runoff coefficients, hydraulic resistance and effective drainage capacity may require tuning.

### **25.19 Model Validation Gap**

- The system needs independent events/datasets for validation.
- Validation should compare predicted flood extent, depth and timing against observed flooding.
- Historical events should be divided appropriately between training/calibration and validation.

### **25.20 Satellite Data Limitations**

- Optical satellite imagery can identify land cover and impervious surfaces but is affected by clouds.
- Satellite revisit frequency may not be sufficient for real-time rainfall/flood prediction.
- Soil-moisture satellite products generally have much coarser spatial resolution than street-level flood predictions.
- Satellite data should therefore complement, not replace, radar, DEM and ground observations.

### **25.21 Urban Feature Gap**

- Roads, buildings, walls, bridges, culverts and underpasses can change water flow.
- These features should be incorporated where they significantly affect surface drainage.
- A basic DEM-only model may miss important urban flow barriers and pathways.

## **25.22 Flood Observation Gap**

- Real-time flood observations may be sparse.
- Crowdsourced reports can be noisy or duplicated.
- Camera-based observations can be affected by visibility and lighting.
- Sensor observations may fail or become unavailable during extreme events.

## **25.23 Routing Gap**

- A route that is geographically shortest may not be flood-safe.
- Flood depth must be translated into road accessibility/risk.
- Different vehicles may have different safe-depth limits.
- Emergency vehicles may require priority routing even when normal roads are restricted.

## **25.24 Infrastructure Failure Gap**

- Extreme flooding can affect pumps, sensors, power supply and communication systems.
- The system should account for unavailable or failed infrastructure where possible.
- Drainage pumps and pumping stations may need separate operational states.

## **25.25 Scalability Gap**

- A high-resolution city-wide simulation can require significant computational resources.
- The system should support processing only the required spatial region when possible.
- GPU/parallel processing may be required for more advanced models.

## **25.26 Data Availability and Licensing**

- Some required datasets may not be freely available.
- Government datasets may have access restrictions or different licensing conditions.
- The implementation should distinguish between publicly available, simulated and restricted datasets.

## **25.27 Missing Data Handling**

- Sensors and external data sources may temporarily fail.
- The system should detect missing or invalid data.
- Appropriate interpolation, persistence or fallback strategies should be defined.

## **25.28 Model Explainability**

- Flood predictions should provide understandable reasons for high-risk areas.
- Example factors can include high rainfall, low elevation, high runoff, drainage overload or blockage.

- This is particularly important for emergency-management decisions.

### **25.29 Forecast Horizon Limitation**

- Prediction uncertainty generally increases with lead time.
- The 15–30 minute forecast may be more reliable than a 3-hour forecast.
- The dashboard should communicate forecast time and confidence appropriately.

### **25.30 Simulation vs Real-World Deployment**

- A hackathon prototype may use historical, simulated or limited real-time data.
- A production system would require continuous operational data feeds, validated hydraulic models and cooperation with relevant authorities.
- Prototype assumptions must therefore be clearly separated from production requirements.

### **25.31 Key Assumption**

The system should not depend on a single dataset. The intended architecture combines multiple sources:

**Radar → Rainfall Nowcast**

**Satellite → Land Cover / Imperviousness / Soil Wetness**

**DEM → Elevation / Surface Flow**

**Drainage GIS → Underground Network**

**Rainfall + Soil + Land Cover → Runoff**

**Runoff + Surface Flow → Drainage Inflow**

**Drainage Network + Hydraulic Capacity → Surcharge / Overflow**

**Flood Model → Water Depth + Extent + Timing**

**Flood Prediction + Road Network → Safe Routing**

**Current Flood Observations → Model Correction / Calibration**

### **25.32 Most Critical Gaps for the Prototype**

For the first working prototype, the most important gaps to solve are:

1. **High-resolution rainfall/radar data**

2. **Rainfall nowcasting**
3. **DEM and surface-flow modelling**
4. **Land-cover/imperviousness**
5. **Soil moisture / antecedent rainfall**
6. **Surface-to-drain connectivity**
7. **Drainage network data**
8. **Hydraulic capacity estimation**
9. **Blockage/effective-capacity estimation**
10. **Current flood-state observations**
11. **Historical flood data for validation**
12. **Street-level flood-depth estimation**
13. **Real-time processing**
14. **Uncertainty estimation**
15. **Flood-aware routing**

These gaps should be treated as engineering and data challenges rather than reasons to exclude the corresponding components from the architecture.